

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

STUDYCOMPASS

Context-Aware Study Resource Recommender Using Learning Patterns

---

Name	**:**	Rajavarshini C
Registration Number	**:**	23MIC0142
Branch & Programme	**:**	Integrated M.Tech CSE
Course Code	**:**	CSI3007
Course Name	**:**	Advanced Python Programming
Faculty Name	**:**	Dr. Sharmila Banu K
Project Domain	**:**	Personalized Learning and Educational Resource Recommendation
Project Type	**:**	AI-Powered Web-Based Application
Programming Language	**:**	Python
Frontend Framework	**:**	Streamlit
Backend Framework	**:**	FastAPI
Frontend Deployment	**:**	Streamlit Community Cloud
Backend Deployment	**:**	Render

MOTIVATION

The motivation behind developing **StudyCompass – Context-Aware Study Resource Recommender Using Learning Patterns** was to explore the development of a practical Python-based application that addresses an interesting aspect of digital learning. The increasing availability of educational content through online platforms, articles, video tutorials, documentation, and practice websites creates opportunities for students to learn independently. However, identifying suitable resources from the large amount of available content can sometimes be challenging.

The project was undertaken with the objective of developing a web-based application that could make educational resource discovery more organized and personalized. The idea evolved into a context-aware recommendation system that considers different learning-related factors, including the selected topic, difficulty level, preferred learning format, and previous academic performance. These factors were incorporated to make the resource selection process more relevant to individual student requirements.

The development of StudyCompass also provided an opportunity to explore Python programming, web application development, API integration, recommendation algorithms, and AI-assisted functionalities. By combining external resource retrieval, a locally maintained resource library, personalized scoring, and analytical visualizations, the project demonstrates how these technologies can be integrated into a single application. Overall, the motivation was to transform an initial project idea into a functional learning resource recommendation platform while gaining practical experience in designing and implementing a real-world Python application.

EXISTING GAP / EVENT THAT MOTIVATED THE PROJECT / USP

1. Existing Gap

The availability of educational resources through online platforms has increased significantly, providing students with access to articles, video tutorials, documentation, courses, and practice materials. However, students often need to search across multiple platforms to identify resources suitable for their learning requirements.

Traditional resource-searching approaches primarily depend on keywords and may not adequately consider individual learning factors such as difficulty level, preferred learning format, previous academic performance, and learning goals. Additionally, educational

resources retrieved from different sources may vary in relevance and quality, making it difficult to identify the most suitable materials.

Another limitation is that students may receive resource links without sufficient explanations about why those resources are relevant to their selected topic or learning requirements. These limitations create an opportunity for a more personalized and organized educational resource recommendation system.

2. Proposed Solution

To address these limitations, StudyCompass – Context-Aware Study Resource Recommender Using Learning Patterns was developed as a web-based application that provides personalized educational resource recommendations.

The system collects information such as the student's selected topic, difficulty level, preferred learning format, previous academic performance, and learning goal. It processes this information to understand the student's learning context and uses AI-assisted personalization to identify relevant search requirements.

StudyCompass retrieves educational resources from multiple sources, including the DEV Community API, Wikipedia API, a locally maintained JSON resource library, and the YouTube Data API when configured. It also supports optional integration with an external partner API.

The retrieved resources are filtered and processed before being evaluated by the Python-based recommendation engine. Google Gemini is the primary AI provider used to interpret student learning context and identify relevant search requirements, while Groq serves as an alternative AI provider. The interpreted information is passed to the Python recommendation engine, which evaluates and ranks the retrieved resources using its scoring mechanism.

The final recommendations are presented through an interactive Streamlit interface, along with relevant resource information, recommendation explanations, and analytical visualizations.

3. Unique Selling Proposition (USP)

The main USP of StudyCompass is its ability to consider multiple learning-related factors rather than relying only on topic-based keyword searching.

Its key distinguishing features include:

1. **Context-Aware Personalization:** Considers the student's topic, difficulty level, preferred learning format, previous performance, and learning goal.
2. **AI-Assisted Learning-Context Interpretation:** Uses Google Gemini as the primary AI provider and Groq as an alternative provider to interpret student learning requirements and identify relevant search keywords and learning preferences.

3. **Python-Based Recommendation Ranking:** Evaluates and ranks retrieved educational resources using a Python-based scoring mechanism that considers relevant resource attributes and student learning requirements.
4. **Multiple Resource Sources:** Combines live educational content from external APIs with a locally maintained collection of curated resources.
5. **Recommendation Explanations:** Presents explanations to help students understand the relevance of recommended resources.
6. **Interactive Dashboard:** Provides an organized interface with recommendation details, filtering options, summary metrics, and analytical visualizations.
7. **API Accessibility:** Includes a FastAPI service that allows external applications to access the recommendation functionality through REST endpoints.

Through these features, StudyCompass aims to make educational resource discovery more personalized, organized, and accessible.

SYSTEM ARCHITECTURE

1. Architecture Overview

StudyCompass follows a modular architecture that integrates user interaction, input validation, learning-context processing, AI-assisted personalization, resource retrieval, Python-based recommendation ranking, and result presentation.

The application consists of two main entry points: a Streamlit-based web interface and a FastAPI-based REST service. Both entry points use a shared Python recommendation pipeline to maintain consistency in the core recommendation functionality.

The system collects student learning information, validates the submitted inputs, and processes the information to understand individual learning requirements. Google Gemini is used as the primary AI provider, while Groq serves as an alternative provider for learning-context interpretation. The interpreted information supports resource discovery by identifying relevant search requirements.

The application retrieves educational resources from multiple sources, processes and filters the retrieved content, and applies Python-based scoring to evaluate and rank suitable resources. The final recommendations, along with relevant explanations and analytical visualizations, are presented through the Streamlit interface. External applications can access recommendation results through the FastAPI REST service in JSON format.

2. Major Architectural Components

1. User Interface Layer

The application uses Streamlit to provide an interactive web-based interface. Students can enter their study topic, select their difficulty level, choose their preferred learning format, provide their previous academic performance, and specify their learning goal.

2. Input Validation and Learning Context Processing

The Python validation module processes the information entered by the student. It validates the input values, normalizes the learning preferences, and creates a structured student context for subsequent recommendation processing.

3. AI-Based Personalization Layer

The AI personalization module uses Google Gemini as its primary AI provider and Groq as an alternative provider. It interprets the student's learning context and helps identify relevant search keywords, difficulty levels, and learning preferences. Rule-based context processing provides a fallback mechanism when AI-based personalization is unavailable.

4. Resource Retrieval Layer

The system retrieves educational resources from multiple sources, including:

- **DEV Community API:** Retrieves programming-related articles.
- **Wikipedia API:** Retrieves general educational reference articles.
- **Local JSON Resource Library:** Provides a collection of curated educational resources.
- **YouTube Data API:** Retrieves video resources when a valid API key is configured. A YouTube search-page fallback is used when direct API retrieval is unavailable.
- **Optional Partner API:** Supports communication with an external educational application when the partner integration is configured.

5. Recommendation and Ranking Layer

The recommendation engine processes the educational resources retrieved from different sources and prepares suitable candidates for evaluation. It removes duplicate entries, evaluates topic relevance, and processes resource information according to the student's learning requirements.

The AI personalization component uses Google Gemini as its primary provider and Groq as an alternative provider to interpret the student's learning context. The interpreted information includes relevant topics, estimated learning needs, suggested difficulty levels, recommended learning formats, learning strategies, and search keywords.

The Python-based recommendation engine then evaluates the retrieved resources using its scoring mechanism. It considers relevant resource attributes and student preferences to calculate recommendation scores and arrange the resources in ranked order.

The final ranked resources are presented through the Streamlit interface. Each recommendation includes relevant information such as its title, resource type, source, score, and explanation. The application also provides resource exploration, filtering, summary metrics, and analytical visualizations.

6. Output and Analytics Layer

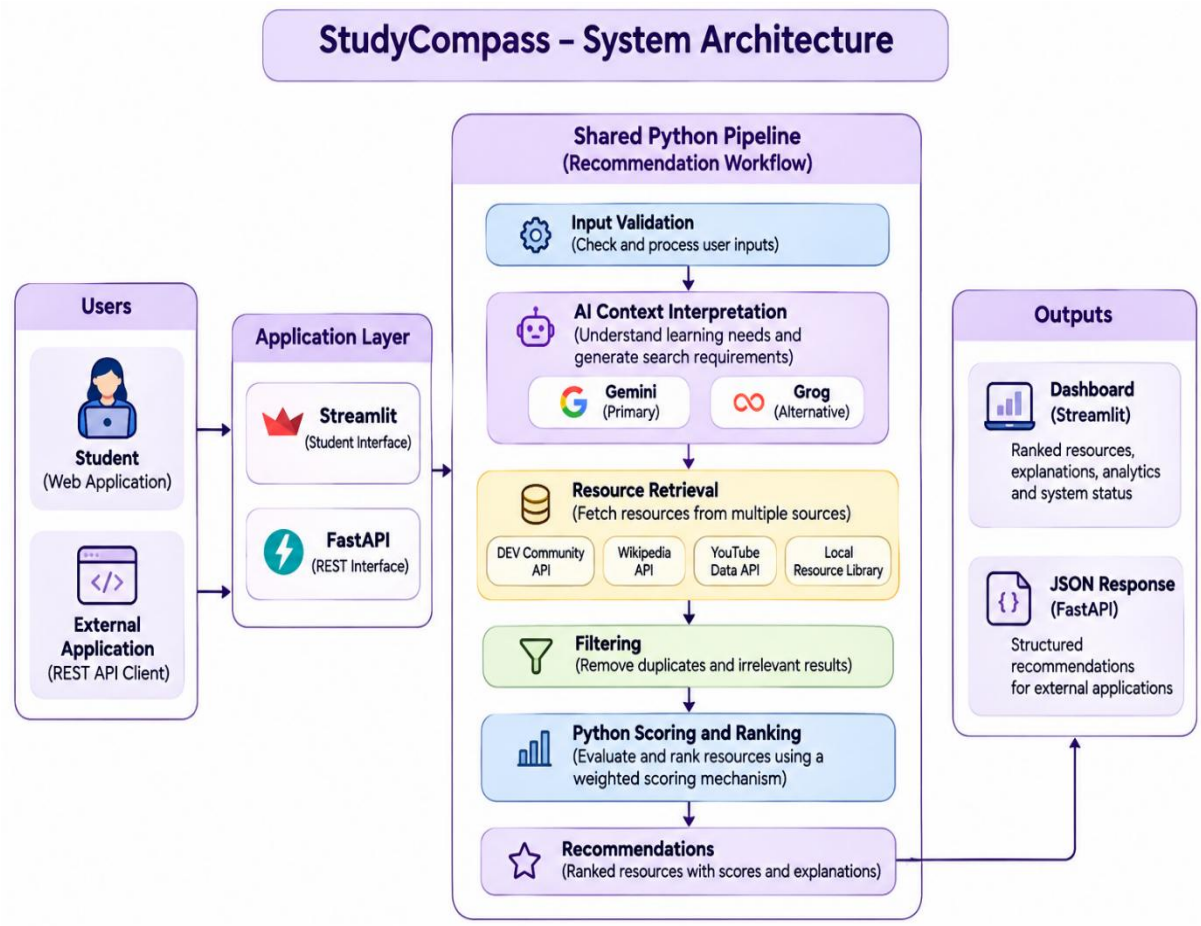
The ranked educational resources are displayed through the Streamlit interface. Each recommendation includes relevant information such as its title, resource type, source, score, and explanation. The application also provides resource exploration, filtering, summary metrics, and analytical visualizations.

7. REST API Layer

The FastAPI service exposes REST endpoints for health checking and recommendation requests. It includes a /api/recommendations endpoint through which external applications can submit student learning information and receive recommendation results in JSON format. The service uses the shared Python recommendation pipeline to process these requests.

3. Architecture Diagram

Figure 1 illustrates the overall system architecture of StudyCompass and the interaction between its major components.



4. Deployment Architecture

The Streamlit web application is deployed using Streamlit Community Cloud, providing access to the student-facing interface. The FastAPI REST service is deployed separately on Render, where it exposes recommendation endpoints for external API access and optional partner integration.

Both services reuse the shared Python recommendation pipeline, ensuring that the core recommendation logic remains consistent across the web application and REST API.

The deployment architecture separates the frontend and backend services while allowing them to perform their respective functions through the shared recommendation system.

COMPLETE TECH STACK INFORMATION

1. Programming Language

Python is the primary and only programming language used in the development of StudyCompass. It handles input validation, learning-context processing, API communication, AI integration, resource retrieval, recommendation ranking, backend operations, and analytical processing.

2. Frontend Technology

Streamlit is used to develop the interactive web-based user interface. It allows students to enter their learning requirements, select preferred difficulty levels and resource formats, provide previous academic scores, and view personalized study recommendations. It also supports the display of recommendation details, analytical metrics, and visualizations.

3. Backend Technology

FastAPI is used to develop the REST API service. It provides endpoints for health checking, recommendation requests, and communication with external partner applications. **Uvicorn** is used as the ASGI server to run the FastAPI application.

4. Artificial Intelligence and LLM Integration

StudyCompass integrates two large language model providers to support AI-assisted interpretation of student learning requirements.

- **Google Gemini:** The primary AI provider used to interpret student learning contexts and identify relevant search requirements. It processes information such as the selected topic, difficulty level, preferred learning formats, previous academic score, and learning goal. It returns structured information containing interpreted topics, estimated learning needs, suggested difficulty levels, recommended formats,

learning strategies, reasoning summaries, and relevant search keywords. The project uses the official google-genai Python SDK.

- **Groq:** Configured as an alternative AI provider when Gemini is unavailable or fails. It performs the corresponding learning-context interpretation through its OpenAI-compatible Chat Completions API using Python HTTP requests.

The interpreted information from these providers is passed to the Python recommendation pipeline, where retrieved educational resources are evaluated and ranked using the application's scoring mechanism.

5. Libraries and Dependencies

Library / Framework	Purpose
Streamlit	Development of the interactive web interface
Requests	HTTP communication with external APIs, including Groq
Pandas	Data processing and analytical operations
python-dotenv	Loading environment variables from a .env file
google-genai	Integration with the Google Gemini API
FastAPI	Development of REST API endpoints
Uvicorn	Running the FastAPI backend server
HTTPX	HTTP client library included in the project dependencies
Pytest	Automated testing of project components

6. External APIs and Data Sources

The application uses the following external services and resource sources:

1. **DEV Community API:** Retrieves live programming-related articles based on topic-derived search tags.
2. **Wikipedia API:** Retrieves general educational reference articles related to the student's selected topic.
3. **YouTube Data API:** Retrieves video recommendations when a valid YouTube API key is configured. A YouTube search-page fallback is provided when direct API retrieval is unavailable.

4. **Google Gemini API:** Provides the primary AI-assisted learning-context interpretation functionality. It identifies relevant learning requirements, suggested difficulty levels, learning formats, and search keywords.
5. **Groq API:** Provides an alternative AI service for learning-context interpretation when the primary Gemini provider is unavailable or encounters an error.
6. **Partner REST API:** Supports optional communication with an external educational application to exchange student learning information and educational resource recommendations.

The resources retrieved from these sources are processed through the shared Python recommendation pipeline. The Python-based scoring mechanism evaluates and ranks the retrieved resources before presenting the final recommendations.

7. Local Data Storage

A local JSON resource library, sample_resources.json, stores **34 curated educational resources**. It provides an additional source of study materials and helps the application maintain a resource collection independently of live API retrieval.

8. Development Environment and Testing Tools

- **Visual Studio Code (VS Code):** Used as the development environment for writing, organizing, and debugging the Python application.
- **Pytest:** Used to execute automated tests and verify the functionality of the application components.

9. Deployment Platforms

- **Streamlit Community Cloud:** Used to host the student-facing Streamlit web application.
- **Render:** Used to deploy the FastAPI backend service and make its REST endpoints accessible externally.

DATASETS USED

1. Overview

StudyCompass does not rely on a conventional machine-learning training dataset or externally downloaded datasets such as CSV, Excel, or Kaggle datasets. Instead, the

application uses a combination of a locally maintained JSON resource library and dynamically retrieved educational content from external APIs.

The system collects student learning preferences and performance information as input during application usage. These inputs are processed to personalize educational resource recommendations.

2. Local JSON Resource Library

The project includes a locally maintained JSON file named sample_resources.json, which contains 34 curated educational resources. This library acts as an additional source of study materials and supports resource recommendations independently of live API retrieval.

The resource collection includes the following categories:

Resource Category	Number of Resources
Documentation	11
Practice Questions	7
Online Courses	6
Video Tutorials	6
Articles	4
Total	**34**

Each resource contains relevant information such as its title, URL, resource type, source, difficulty level, tags, description, and quality tier. These attributes help the application organize and evaluate resources according to student learning requirements.

3. External Data Sources

In addition to the local JSON resource library, StudyCompass retrieves educational content from external APIs.

1. DEV Community API

Provides access to programming-related articles based on relevant topics and search tags.

2. Wikipedia API

Retrieves general reference articles that help students explore educational topics.

3. YouTube Data API

Provides video search results when a valid API key is configured. When direct API retrieval is unavailable, the application can provide a YouTube search-page fallback.

4. Optional Partner API

Supports the retrieval and exchange of educational resource information through an external application when the partner integration is configured.

4. Student Input Data

StudyCompass also collects student-specific learning information during application usage. This information includes:

- Selected study topic.
- Preferred difficulty level.
- Preferred learning format.
- Previous academic performance or assessment score.
- Learning goal.

These inputs are used to understand the student's learning context and personalize the recommendation process. They are not treated as a separate pre-existing training dataset.

5. Dataset Usage in the Recommendation Process

The locally stored resources and externally retrieved educational content are combined and processed by the recommendation pipeline. The system filters and organizes the retrieved resources before applying AI-assisted learning-context interpretation.

Google Gemini, as the primary AI provider, and Groq, as the alternative provider, help interpret student learning requirements and identify relevant search information. The interpreted information is then passed to the Python recommendation engine.

The Python-based scoring mechanism evaluates the retrieved resources according to relevant resource attributes and student learning preferences. It calculates recommendation scores and ranks the resources to identify suitable educational materials.

The final results are displayed through the Streamlit interface with relevant resource details, recommendation scores, and explanations.

Overall, StudyCompass uses a resource-based and API-driven approach instead of training a machine-learning model on a conventional dataset.

PROMPTS USED FOR WEBSITE CONSTRUCTION

1. Introduction

ChatGPT was used as an AI-assisted development tool throughout the development of StudyCompass. It helped in planning the application, designing its architecture, developing frontend and backend components, integrating external APIs, implementing recommendation functionality, debugging errors, and improving the overall application.

A series of prompts were prepared to obtain technical guidance and generate implementation suggestions during different development stages. The following are representative prompts based on the functionalities and technologies used in the project.

2. Project Planning and Requirement Analysis

Prompt 1: "Suggest a Python-based project idea for developing a context-aware study resource recommendation system. The application should consider the student's topic, difficulty level, preferred learning format, previous academic performance, and learning goals. Suggest suitable technologies and explain the major functionalities."

Purpose: To establish the project concept, identify its objectives, and define the major application requirements.

3. System Architecture Design

Prompt 2: "Design a modular architecture for a Context-Aware Study Resource Recommender using Python, Streamlit, and FastAPI. Include input validation, learning-context processing, AI personalization, external resource retrieval, recommendation ranking, and result presentation. Explain how the frontend and backend can share a common recommendation pipeline."

Purpose: To understand the system structure and organize the application into separate functional components.

4. Frontend Development

Prompt 3: "Develop an interactive Streamlit interface for a study resource recommendation application named StudyCompass. Include fields for entering a study topic, selecting difficulty level, choosing preferred learning formats, entering previous academic performance, and specifying learning goals. Display recommendations in an organized dashboard with resource details and analytical visualizations."

Purpose: To develop the student-facing interface and improve the presentation of recommendations.

5. Frontend Development

Prompt 4: "Create a FastAPI backend for a context-aware study resource recommendation system. Include a health-check endpoint and a recommendation endpoint that accepts student learning information and returns recommended educational resources in JSON format. Structure the backend so that it can reuse a shared Python recommendation pipeline."

Purpose: To develop the REST API service and support external application communication.

6. API Integration and Personalization

Prompt 5: "Integrate the Google Gemini API into a Python-based study recommendation application. Use student learning context to generate relevant search keywords, identify suitable difficulty levels, and personalize educational resource recommendations. Include appropriate error handling."

Purpose: To implement AI-based learning-context interpretation and personalization.

Prompt 6: "Implement an alternative LLM provider using the Groq API for a Python recommendation system. Configure it as a backup when the primary Gemini provider is unavailable. Maintain a consistent input and output structure between the two providers."

Purpose: To support alternative AI-based processing and improve the application's ability to handle provider failures.

7. External API Integration

Prompt 7: "Integrate the DEV Community API and Wikipedia API into a Python-based educational resource recommendation application. Retrieve resources relevant to the student's selected topic, process the API responses, and organize the results into a consistent resource format."

Purpose: To retrieve educational content dynamically from external sources.

Prompt 8: "Add YouTube Data API support to the study resource recommendation system. Retrieve relevant educational videos when an API key is configured and provide a suitable fallback when direct API retrieval is unavailable."

Purpose: To support video resource retrieval and handle missing API configuration.

8. Recommendation and Ranking Mechanism

Prompt 9: "Develop a context-aware recommendation pipeline that processes resources retrieved from different sources. Include duplicate removal, relevance evaluation, AI-assisted context interpretation, resource filtering, Python-based weighted scoring, and recommendation explanations."

Purpose: To implement the resource-processing and recommendation-ranking functionality.

9. Debugging and Testing

Prompt 10: "Review the Python code of the study resource recommendation application, identify potential errors, and suggest corrections. Check API communication, input validation, error handling, fallback mechanisms, and the consistency of recommendation outputs. Suggest suitable Pytest test cases."

Purpose: To identify implementation errors, improve reliability, and verify application functionality.

10. Deployment Support

Prompt 11: "Explain how to deploy a Streamlit frontend using Streamlit Community Cloud and a FastAPI backend using Render. Include the necessary environment variables, dependency configuration, backend URL setup, and steps to verify that both services are accessible."

Purpose: To obtain deployment guidance and configure the frontend and backend services for online access.

11. Summary

ChatGPT supported the development process by providing implementation guidance, code suggestions, debugging assistance, architectural recommendations, and deployment instructions. The generated suggestions were adapted and incorporated into the application during development. This AI-assisted approach helped organize the development process and integrate multiple Python technologies into the StudyCompass application.

API INTEGRATION

1. Introduction

API integration is an important part of StudyCompass because it enables the application to retrieve educational resources from external platforms, communicate with AI services, and exchange recommendation information with other applications.

The project uses REST-based HTTP communication and JSON data exchange to connect different services. Python libraries such as Requests and HTTPX are included in the project dependencies to support HTTP communication.

2. Google Gemini API Integration

The Google Gemini API is integrated as the primary AI service in StudyCompass. It is accessed using the official google-genai Python SDK.

The application sends structured student learning information, including the selected topic, difficulty level, preferred learning formats, previous academic score, and learning goal.

Gemini interprets the learning context and returns structured JSON information containing interpreted topics, estimated learning needs, suggested difficulty levels, recommended formats, learning strategies, and relevant search keywords.

The returned information is validated and used by the recommendation pipeline to support personalized resource discovery.

3. Groq API Integration

Groq is integrated as an alternative AI provider when the primary Gemini service is unavailable or encounters an error.

The application communicates with Groq through its OpenAI-compatible Chat Completions REST API using the Python Requests library. Authentication is performed using an API key.

This integration provides an alternative mechanism for AI-based learning-context processing and recommendation operations.

4. DEV Community API Integration

StudyCompass integrates the DEV Community (Forem) API to retrieve programming-related articles.

The application derives relevant search tags from the student's selected topic and sends HTTP GET requests to the DEV Community articles endpoint.

The retrieved JSON responses are processed to extract information such as:

- Article title.
- Article URL.
- Author information.
- Tags.
- Description.
- Publication date.
- Reading time.
- Reactions and comments.

The retrieved articles are converted into a common resource format before being passed to the recommendation pipeline.

5. Wikipedia API Integration

The Wikipedia API is integrated to retrieve general educational reference articles based on the student's selected topic.

The application sends a search request to the Wikipedia API and processes the returned JSON response. Relevant information, including article titles, URLs, and introductory descriptions, is extracted and converted into the application's resource format.

These reference articles provide additional educational material, particularly when students need introductory information about unfamiliar topics.

6. YouTube Data API Integration

The YouTube Data API is used to retrieve educational video resources related to the selected topic.

The application sends a search request to the YouTube Data API using the configured API key and processes the returned video information.

When a valid API key is unavailable or direct API retrieval cannot be used, the application provides a YouTube search-page fallback so that students can still explore relevant videos.

7. Partner API Integration

StudyCompass supports API communication with an external educational application through a partner integration.

The integration enables learning-related information to be exchanged between the applications. Student information such as the selected course, topic, difficulty level, assessment score, completed lessons, learning progress, and preferred resource type can be transferred to the StudyCompass recommendation service.

The FastAPI backend exposes a partner-facing endpoint:

Endpoint: POST /api/recommendations

The received information is processed and converted into the format required by the shared recommendation pipeline. StudyCompass then generates relevant educational resource recommendations and returns the results in JSON format.

The application also supports outgoing API communication through a configurable partner client, allowing it to send student learning context to an external application and process the returned educational resources.

The partner API integration was tested successfully, demonstrating communication and data exchange between the applications.

8. FastAPI REST API Integration

StudyCompass uses FastAPI to expose its recommendation functionality through REST endpoints.

The backend provides the following endpoints:

HTTP Method	Endpoint	Purpose
GET	/health	Checks the availability of the backend service
POST	/recommend	Accepts student learning information and generates recommendations
POST	/api/recommendations	Accepts partner application requests and returns recommendations

The recommendation endpoints accept JSON input, validate the received information, execute the shared Python recommendation pipeline, and return the processed results as JSON responses.

9. API Authentication and Configuration

API credentials and service configurations are managed through environment variables rather than being directly embedded in the application code.

The project uses a .env configuration file during local development and environment-variable settings during deployment.

The API integrations use the following configuration:

- Google Gemini API key.
- Groq API key.
- YouTube Data API key.
- Partner API URL and optional API key.

This approach helps protect sensitive credentials and allows external service configurations to be changed without modifying the main application code.

10. Summary

The API integrations enable StudyCompass to combine AI-based learning-context processing, live educational resource retrieval, and communication with external applications. By integrating Google Gemini, Groq, DEV Community, Wikipedia, YouTube, and FastAPI, the application provides a connected and extensible environment for personalized educational resource recommendations.

USERS, INPUTS AND OUTPUTS

1. Intended Users

StudyCompass is primarily designed for students who want to discover educational resources according to their learning requirements. It helps students find relevant study materials by considering their selected topic, current knowledge level, preferred learning formats, previous academic performance, and learning goals.

The application also supports external applications through its FastAPI-based REST interface. These applications can submit student-learning information and receive resource recommendations in a structured JSON response. This enables StudyCompass to function both as an independent web application and as a recommendation service that can be integrated with other learning platforms.

2. Input Parameters

The application collects different input parameters to understand the learner's requirements before generating recommendations.

A. Student Interface Inputs

Input Parameter	Description
Topic or Subject	The topic for which the student needs learning resources.
Current Difficulty Level	The student's selected level: Beginner, Intermediate, or Advanced.
Preferred Formats	The preferred learning formats, such as Video Tutorials, Articles, Documentation, Practice Questions, and Online Courses.
Previous Score	The student's previous academic or assessment score, entered between 0 and 100.
Learning Goal	The student's objective, such as understanding fundamentals, practising problem-solving, or preparing for an examination.
Additional Context	Optional information describing specific learning difficulties or requirements.

B. External Application API Inputs

The FastAPI interface also accepts structured information from an external application. The supported parameters include:

- Course and topic details.
- Difficulty level.
- Assessment score.
- Number of completed lessons.
- Learning progress.
- Preferred resource type.
- Requested number of recommendations.
- Optional student identifier.

The API converts the received difficulty and resource-format values into the corresponding formats supported by StudyCompass before processing the recommendation request.

3. System Outputs

After processing the submitted information, StudyCompass generates personalized learning recommendations and presents the results through its web interface.

The major outputs include:

1. **Ranked Learning Resources:** A list of recommended educational resources arranged according to their recommendation scores.
2. **Resource Information:** Each recommendation displays its title, resource format, source, and origin.
3. **Recommendation Score:** A score displayed on a 0–100 scale to indicate the resource's evaluation result.
4. **Explanation:** A short explanation describing why a particular resource was recommended.
5. **Personalized Learning Guidance:** Interpreted topic information, suggested difficulty, and learning strategy when personalization context is available.
6. **Resource Links:** Direct links that allow students to access the recommended learning materials.
7. **Learning Analytics:** Visual summaries of recommendation scores, resource-format distribution, average scores, and evaluated resources.
8. **System Status:** Information about personalization, ranking, live resource retrieval, local fallback, and partner API outcomes.

4. API Response Outputs

For external applications, StudyCompass returns recommendations in JSON format through its FastAPI endpoints. The partner-facing endpoint provides fields such as resource title, URL, resource type, difficulty level, recommendation score, and explanation. It also returns response information such as status, message, and recommendation count.

This structured response allows another application to process and display the recommendations without depending on the Streamlit user interface.

5. Overall Input-Processing-Output Flow

The overall operation follows a simple sequence: the student or external application submits learning-related information; StudyCompass validates and processes the input, retrieves relevant educational resources, and generates recommendations; finally, the system displays the results through the Streamlit interface or returns them as a JSON response through the FastAPI service.

This approach allows StudyCompass to support both direct student interaction and integration with external educational applications.

REFERENCES

[1] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/

[2] Streamlit, “Streamlit Documentation.” [Online]. Available: https://docs.streamlit.io/

[3] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[4] Uvicorn, “Uvicorn Documentation.” [Online]. Available: https://www.uvicorn.org/

[5] Google, “Gemini API Documentation – Google GenAI SDK.” [Online]. Available: https://ai.google.dev/gemini-api/docs

[6] Groq, “Groq API Reference.” [Online]. Available: https://console.groq.com/docs/api-reference

[7] DEV Community, “Forem API Documentation.” [Online]. Available: https://developers.forem.com/api

[8] MediaWiki, “API: Search.” [Online]. Available: https://www.mediawiki.org/wiki/API:Search

[9] Google for Developers, “YouTube Data API – Search: List.” [Online]. Available: https://developers.google.com/youtube/v3/docs/search/list

[10] pandas Development Team, “pandas Documentation.” [Online]. Available: https://pandas.pydata.org/docs/

[11] Python Software Foundation, “Requests: HTTP for Humans.” [Online]. Available: https://requests.readthedocs.io/

[12] pytest Development Team, “pytest Documentation.” [Online]. Available: https://docs.pytest.org/

[13] Render, “Deploy a FastAPI App.” [Online]. Available: https://render.com/docs/deploy-fastapi

[14] P. Melville, R. J. Mooney, and R. Nagarajan, “Content-Boosted Collaborative Filtering for Improved Recommendations,” in *Proceedings of the 18th National Conference on Artificial Intelligence (AAAI)*, 2002, pp. 187–192. [Online]. Available: https://mlanthology.org/aaai/2002/melville2002aaai-content/

[15] D. Roy and M. Dutta, “A systematic review and research perspective on recommender systems,” *Journal of Big Data*, vol. 9, article 59, 2022. [Online]. Available: https://doi.org/10.1186/S40537-022-00592-5